

QVerse — Week 10: Oracles and the Deutsch–Jozsa Algorithm

Theory Booklet | Introduction to Quantum Computing & Programming

QVerse

2026

How this booklet fits the course

This theory booklet is designed to be read alongside `10_oracles_deutsch_jozsa.ipynb`. It contains the conceptual and mathematical material that should be explained during Week 10; the notebook then turns those ideas into experiments, circuit construction, and exercises.

Learning objectives

- Recognize state preparation–oracle–interference–measurement structure.
- Understand an oracle as reversible problem encoding.
- Implement simple constant and balanced Deutsch–Jozsa oracles.
- Interpret the final measurement pattern.

Recommended workflow

1. Read the relevant theory section.
2. Predict the result of the corresponding notebook experiment.
3. Run the notebook.
4. Explain any difference between the prediction and the output.
5. Complete the core exercises without treating the solution notebook as a substitute for reasoning.

1. What is an oracle?

In quantum algorithms, an **oracle** is a circuit that encodes a function or a problem instance in a reversible way.

For a Boolean function

$$f : \{0, 1\}^n \rightarrow \{0, 1\},$$

a standard reversible oracle acts as

$$U_f|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle,$$

where $\oplus$ is XOR.

This construction is reversible because knowing x and the output lets us undo the operation by applying the same XOR again.

The oracle model lets us analyze how many times an algorithm must query a function without focusing on every low-level gate used to implement that function.

2. Reversibility

Ordinary classical Boolean functions need not be reversible. For example, many different inputs can produce the same output.

Quantum gates, however, are unitary and therefore reversible.

The standard oracle construction preserves the input x and XORs the function value into an auxiliary target. This embeds a classical function into a reversible quantum operation.

Reversibility is a structural requirement of closed-system quantum circuit evolution.

3. The Deutsch-Jozsa promise problem

The Deutsch-Jozsa problem assumes that the unknown Boolean function is promised to be either:

- **constant:** $f(x)$ has the same value for every input;
- **balanced:** $f(x) = 0$ for exactly half of all inputs and $f(x) = 1$ for the other half.

The task is to determine which case holds.

The promise is important. A general arbitrary Boolean function could be neither constant nor balanced.

4. Classical deterministic query complexity

For an n -bit input, there are

$$N = 2^n$$

possible inputs.

In the worst case, a deterministic classical algorithm may need to evaluate

$$2^{n-1} + 1$$

inputs to distinguish a constant function from a balanced one with certainty.

Why? After observing 2^{n-1} identical outputs, the function could still be constant or could be balanced with all opposite outputs hidden among the remaining inputs.

The quantum algorithm uses one oracle query under the standard query-complexity model.

This is an exponential separation in exact deterministic query complexity for this promise problem. It is mainly pedagogical rather than a practical modern speedup.

5. State preparation

The standard algorithm uses n input qubits initialized to

$$|0\rangle^{\otimes n}$$

and one auxiliary qubit initialized to

$$|1\rangle.$$

Applying Hadamards gives

$$\frac{1}{\sqrt{2^n}} \sum_x |x\rangle \otimes |-\rangle,$$

where

$$|-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}.$$

The input register now contains a uniform superposition over all 2^n inputs.

6. Phase kickback

Because the auxiliary qubit is $|-\rangle$,

$$X|-\rangle = -|-\rangle.$$

Therefore the oracle action becomes

$$U_f|x\rangle|-\rangle = (-1)^{f(x)}|x\rangle|-\rangle.$$

The function value has been converted into a phase:

$$|x\rangle \rightarrow (-1)^{f(x)}|x\rangle.$$

This is called **phase kickback**.

The auxiliary qubit factors out of the problem after the oracle. The useful information is now encoded in the relative phases of the input amplitudes.

7. Interference after the oracle

After the oracle, the input state is

$$\frac{1}{\sqrt{2^n}} \sum_x (-1)^{f(x)} |x\rangle.$$

Applying $H^{\otimes n}$ causes these amplitudes to interfere.

The amplitude of the all-zero state is

$$\frac{1}{2^n} \sum_x (-1)^{f(x)}.$$

If f is constant, all signs are the same, so the magnitude of the sum is 1 after normalization.

If f is balanced, half the terms are +1 and half are -1, so they cancel and the amplitude of $|00 \dots 0\rangle$ is zero.

Therefore:

- measure all zeros $\Rightarrow$ constant;
- measure anything else $\Rightarrow$ balanced.

In the ideal algorithm, the classification is deterministic.

8. The algorithmic pattern

Deutsch-Jozsa is valuable because it makes a general quantum-algorithm structure explicit:

$$\text{state preparation} \rightarrow \text{oracle} \rightarrow \text{interference} \rightarrow \text{measurement}.$$

The oracle does not directly print the answer. It alters phases.

The final Hadamards convert the global phase pattern into a measurable property.

This architecture reappears in more sophisticated algorithms.

9. Constant oracles

Two constant functions are possible:

$$f(x) = 0 \quad \forall x$$

and

$$f(x) = 1 \quad \forall x.$$

For $f(x) = 0$, the oracle may effectively do nothing to the target.

For $f(x) = 1$, the oracle flips the target for every input.

Because the target is $|-\rangle$, the latter introduces an overall phase of -1 , which is global for the input state and therefore does not alter the final classification.

Both constant cases return the all-zero input register.

10. Balanced teaching oracles from linear masks

A convenient balanced family is

$$f_a(x) = a \cdot x \pmod{2},$$

where a is a nonzero binary mask and the dot product is modulo 2.

For nonzero a , exactly half of all bit strings have parity 0 with respect to the mask and half have parity 1.

These oracles can be constructed using CNOT operations from selected input qubits to the auxiliary target.

This is why the notebook can generate many different balanced examples while reusing the same Deutsch-Jozsa driver.

11. Why the oracle must be interpreted carefully

An oracle is not magical. In a real application, someone must build a circuit that implements the desired function.

Query complexity often treats the oracle as one black-box call. Gate complexity counts the physical resources inside the oracle as well.

Therefore claims of speedup should specify the cost model being used.

12. Common misconceptions

“The quantum computer evaluates every $f(x)$ and reads all answers”

The state contains a coherent encoding over inputs, but measurement reveals only limited classical data. Interference extracts the global property.

“The oracle is the algorithm”

The oracle encodes the problem. The surrounding state preparation and interference are what convert that encoding into an answer.

“Balanced means random-looking”

Balanced has a precise definition: exactly half of the inputs map to 0 and half to 1.

“Deutsch-Jozsa is a broadly useful practical algorithm”

Its main importance here is conceptual: phase kickback, oracle design, and interference.

13. What to know before the notebook

You should be able to explain:

$$U_f|x, y\rangle = |x, y \oplus f(x)\rangle,$$

$$U_f|x\rangle|-\rangle = (-1)^{f(x)}|x\rangle|-\rangle,$$

and why the all-zero measurement distinguishes constant from balanced functions after the final Hadamards.

Suggested references

- IBM Quantum Learning, *Basics of Quantum Information* and current Qiskit learning material: <https://quantum.cloud.ibm.com/learning/en/courses>
- IBM Quantum Documentation and Qiskit guides: <https://quantum.cloud.ibm.com/docs>
- Quantum Rings, *Quantum 101*: <https://www.quantumrings.com/course/quantum101>
- M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press.

The software-facing details in the practical notebook should be treated as the canonical implementation for this course. The purpose of this booklet is to explain the physics, mathematics, and algorithmic reasoning behind that implementation.